Group members :

Pranav Bavadiya : 2020301458

Om Patel : 202301027

Nauman Shethwala : 202301237

WEEK 11

AIM: a)To Implement in c language the following UNIX commands using system calls i) cat ii) ls iii) Scanning Directories (Ex: opendir(),readdir(),etc.)

PROGRAM AND OUTPUTS

A) cat COMMAND:

Source Code:

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int fd, n;
    char buff[512];

    if (argc != 2) {
        printf("ENTER CORRECT ARGUMENTS\n");
        exit(1);
    }

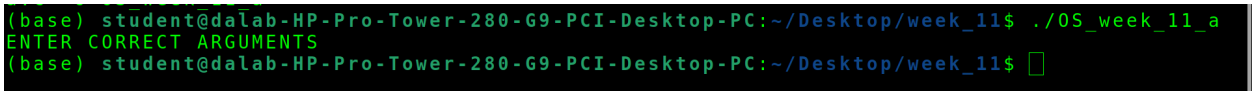
    fd = open(argv[1], O_RDONLY);
    if (fd < 0) {
        printf("ERROR\n");
        return 1;
    }

    while ((n = read(fd, buff, sizeof(buff))) > 0) {
```

```
        write(1, buff, n);
    }

    close(fd);
    return 0;
}
```

OUTPUT



B) ls

Source Code:

```
#include <stdio.h>
#include <dirent.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    DIR *dirop;
    struct dirent *dired;

    if (argc != 2) {
        printf("Invalid number of arguments\n");
        return 1;
    }

    dirop = opendir(argv[1]);
    if (dirop == NULL) {
        printf("Cannot open directory\n");
        return 1;
    }

    while ((dired = readdir(dirop)) != NULL) {
        printf("%10ld %s\n", dired->d_ino, dired->d_name);
    }

    closedir(dirop);
    return 0;
}
```

OUTPUT

```
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/week_11$ gcc OS_week_11_
b.c -o OS_week_11_b
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/week_11$ ./OS_week_11_b
Invalid number of arguments
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/week_11$ ./OS_week_11_b
asdfgh
Cannot open directory
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/week_11$ ./OS_week_11_a
ghfjd
ERROR
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/week_11$
```

C) 5c.c

Source Code:

```
#include <stdio.h>
#include <dirent.h>

int main(void)
{
    struct dirent *de; // Pointer for directory entry
    DIR *dr = opendir("."); // opendir() opens the current directory

    if (dr == NULL) // Check if opendir returns NULL (failed to open directory)
    {
        printf("Could not open current directory\n");
        return 0;
    }

    // Loop through the directory entries using readdir() and print their names
    while ((de = readdir(dr)) != NULL)
    {
        printf("%s\n", de->d_name);
    }

    closedir(dr); // Close the directory when done
    return 0;
}
```

OUTPUT

```
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./11_b.o
studentshare
.local
.profile
.ssh
.bash_logout
.config
.ipython
Videos
.thunderbird
Untitled4.ipynb
Pictures
.bashrc
11_b.o
abc.c
.conda
Desktop
.bash_history
eclipse-workspace
Templates
.anaconda
.dotnet
11_b.c
.continuum
Documents
Untitled2.ipynb
.matlab
.condarc
.ipynb_checkpoints
.dia
.gnupg
202301131
.
Public
.jupyter
.MathWorks
.p2
snap
Downloads
.java
Untitled1.ipynb
netbeans-8.2-linux.sh
..
.pki
.netbeans
Untitled3.ipynb
.adobe
.eclipse
.pgadmin
Anaconda3-2024.02-1-Linux-x86_64.sh
Music
.cache
.swt
Untitled5.ipynb
```

AIM: b)To write a C program to create child process and allow parent process to display “parent” and the child to display “child” on the screen

PROGRAM AND OUTPUTS

8a.c

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
   int pid, pid1, pid2;
   pid = fork();

   if (pid == -1)
   {
      printf("ERROR IN PROCESS CREATION\n");
      exit(0);
   }

   if (pid == 0)
   {
      pid1 = getpid();
      printf("\nThe parent process ID is %d", pid1);
   }
   else
   {
      pid2 = getpid();
      printf("\nThe child process ID is %d\n", pid2);
   }

   return 0;
}
```

OUTPUT

```
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc 11_b.c -o 11_b.o
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./11_b.o

The child process ID is 4196

The parent process ID is 4197(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Deskto
p-PC:~$
```

WEEK 12 LAB REPORT

AIM: Write a C program to simulate disk scheduling algorithms. a) FCFS b) SCAN c) C-SCAN

PROGRAM

FCFS DISK SCHEDULING ALGORITHM:

```
#include <stdio.h>

int main()
{
    int t[20], n, i, tohm[20], tot = 0;
    float avhm;

    printf("Enter the no. of tracks: ");
    scanf("%d", &n);

    printf("Enter the tracks to be traversed:\n");
    for (i = 1; i <= n; i++)
    {
        scanf("%d", &t[i]);
    }

    for (i = 1; i < n; i++)
    {
        tohm[i] = t[i + 1] - t[i];
        if (tohm[i] < 0)
            tohm[i] = -tohm[i];
    }

    for (i = 1; i < n; i++)
    {
```

```
        tot += tohm[i];
    }

    avhm = (float)tot / (n - 1);

    printf("\nTracks traversed\tDifference between tracks\n");
    for (i = 1; i < n; i++)
    {
        printf("%d -> %d\t\t%d\n", t[i], t[i + 1], tohm[i]);
    }

    printf("\nAverage header movements: %f\n", avhm);

    return 0;
}
```

OUTPUT

```
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc fcfs.c -o fcfs
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./fcfs
Enter the no. of tracks: 9
Enter the tracks to be traversed:
55 58 60 70 18 90 150 160 184

Tracks traversed        Difference between tracks
55 -> 58                3
58 -> 60                2
60 -> 70                10
70 -> 18                52
18 -> 90                72
90 -> 150               60
150 -> 160              10
160 -> 184              24

Average header movements: 29.125000
```

PROGRAM

SCAN DISK SCHEDULING ALGORITHM:

```
#include <stdio.h>

int main()
{
    int i, j, sum = 0, n;
    int d[20];
    int disk;       // location of head
    int temp, max;
```

```
    int dloc;      // location of disk in array

    printf("Enter number of locations: ");
    scanf("%d", &n);

    printf("Enter position of head: ");
    scanf("%d", &disk);

    printf("Enter elements of disk queue:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &d[i]);
    }

    // Add head position into array
    d[n] = disk;
    n = n + 1;

    // Sorting disk locations
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (d[i] > d[j])
            {
                temp = d[i];
                d[i] = d[j];
                d[j] = temp;
            }
        }
    }

    max = d[n - 1];

    // Find location of head in sorted array
    for (i = 0; i < n; i++)
    {
        if (disk == d[i])
        {
            dloc = i;
            break;
        }
    }

    // Move towards 0 (left side)
    for (i = dloc; i >= 0; i--)
    {
        printf("%d --> ", d[i]);
    }
```

```
    printf("0 --> ");

    // Move towards right side
    for (i = dloc + 1; i < n; i++)
    {
        printf("%d --> ", d[i]);
    }

    // Total movement (SCAN logic approximation)
    sum = disk + max;

    printf("\nTotal head movement: %d\n", sum);

    return 0;
}
```

OUTPUT

```
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc scan.c -o scan
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./scan
Enter number of locations: 8
Enter position of head: 53
Enter elements of disk queue:
98
183
37
122
14
124
65
67
53 --> 37 --> 14 --> 0 --> 65 --> 67 --> 98 --> 122 --> 124 --> 183 -->
Total head movement: 236
```

PROGRAM

C-SCAN DISK SCHEDULING ALGORITHM:

```
#include <stdio.h>
#include <stdlib.h>  // Needed for abs()

int main()
{
   int queue[20], n, head, i, j, k, seek = 0, max, diff, temp, queue1[20], queue2[20];
   int temp1 = 0, temp2 = 0;
```

```
    float avg;

    // Taking input for the disk range, initial head position, and the queue of requests
    printf("Enter the max range of disk\n");
    scanf("%d", &max);
    printf("Enter the initial head position\n");
    scanf("%d", &head);
    printf("Enter the size of queue request\n");
    scanf("%d", &n);
    printf("Enter the queue of disk positions to be read\n");

    // Separate the requests into two queues: one for positions greater than or equal to the
head
    // and the other for positions less than the head.
    for (i = 0; i < n; i++)
    {
        scanf("%d", &temp);
        if (temp >= head)
        {
            queue1[temp1] = temp;
            temp1++;
        }
        else
        {
            queue2[temp2] = temp;
            temp2++;
        }
    }

    // Sort the first queue (queue1) in ascending order
    for (i = 0; i < temp1 - 1; i++)
    {
        for (j = i + 1; j < temp1; j++)
        {
            if (queue1[i] > queue1[j])
            {
                temp = queue1[i];
                queue1[i] = queue1[j];
                queue1[j] = temp;
            }
        }
    }

    // Sort the second queue (queue2) in descending order
    for (i = 0; i < temp2 - 1; i++)
```

```
    {
        for (j = i + 1; j < temp2; j++)
        {
            if (queue2[i] < queue2[j])  // Change to '<' for descending order
            {
                temp = queue2[i];
                queue2[i] = queue2[j];
                queue2[j] = temp;
            }
        }
    }

    // Merge the two queues into the main queue
    for (i = 0, j = 0; j < temp1; i++, j++)
    {
        queue[i] = queue1[j];
    }
    queue[i] = max;
    i++;

    for (j = 0; j < temp2; j++, i++)
    {
        queue[i] = queue2[j];
    }

    queue[i] = 0; // Add 0 for the final position of the disk head

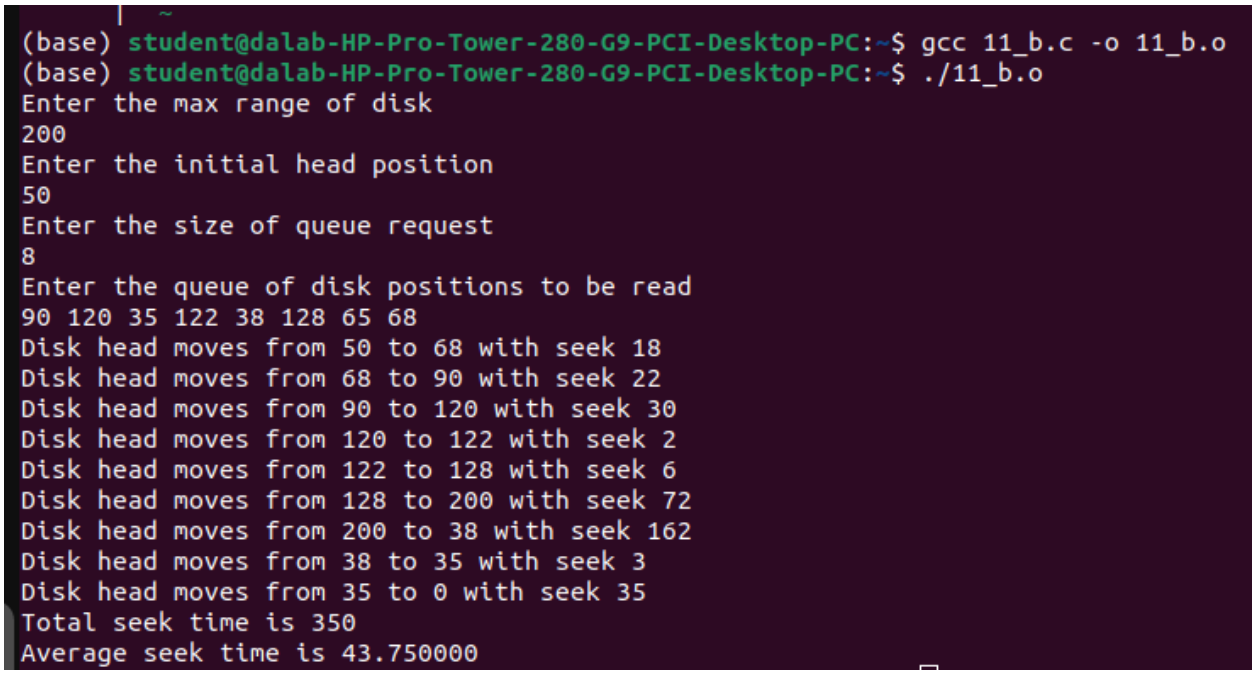
    // Calculate the total seek time
    queue[0] = head; // Set the initial position of the head
    for (j = 0; j < n + 1; j++)
    {
        diff = abs(queue[j + 1] - queue[j]);
        seek += diff;
        printf("Disk head moves from %d to %d with seek %d\n", queue[j], queue[j + 1], diff);
    }

    printf("Total seek time is %d\n", seek);

    // Calculate the average seek time
    avg = (float)seek / n;
    printf("Average seek time is %f\n", avg);

    return 0;
}
```

OUTPUT


(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc 11_b.c -o 11_b.o
(base) student@dalab-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./11_b.o
Enter the max range of disk
200
Enter the initial head position
50
Enter the size of queue request
8
Enter the queue of disk positions to be read
90 120 35 122 38 128 65 68
Disk head moves from 50 to 68 with seek 18
Disk head moves from 68 to 90 with seek 22
Disk head moves from 90 to 120 with seek 30
Disk head moves from 120 to 122 with seek 2
Disk head moves from 122 to 128 with seek 6
Disk head moves from 128 to 200 with seek 72
Disk head moves from 200 to 38 with seek 162
Disk head moves from 38 to 35 with seek 3
Disk head moves from 35 to 0 with seek 35
Total seek time is 350
Average seek time is 43.750000